



SEMESTER 2/20252026

SPECIAL TOPIC IN DATA ENGINEERING (SECP3843)

SECTION 01

Tutorial: The AI Workflow for Data Engineering

LECTURER:

DR ARYATI BINTI BAKRI

NO	NAME	MATRIC NO
1	LIM YU HAN	A23CS0241
2	LING YU QIAN	A23CS0301
3	TEH RU QIAN	A23CS0191

1.0 What is AI-assisted data engineering

For AI-assisted data engineering, developers are moving away from the traditional script-based approach and into an agentic and automated approach with AI-based tools integrated directly into the developer environment.

Comprehensive Concept: AI workflows are not just about copying and pasting code snippets; they involve the use of autonomous agents, such as Codex, that directly interact with the file system, command lines, and data warehouse.

Modern Data Engineering: Modern data architecture often involves a lot of boilerplate code and configuration when establishing strong ETL pipelines. AI agents can help with repetitive work, and engineers can spend significantly more time on data modeling, data granularity, and architecture design, instead of debugging syntax.

Examples in Practice: The AI can auto-create staging layers, create complex SQL transformations, create table relationships, and even create full project documentation dynamically based on natural language instructions.

2.0 What AI agent is used in the tutorial?

In the tutorial, Codex is an embedded co-developer that helps navigate the project architecture to automate and carry out fundamental data engineering tasks.

Role in Automation: Codex automates the Git version control workflow without hassle. It can independently create feature branches, stage modified dbt files, commit changes, push to a remote repository, and add a pull request (PR) to the repository, all without the intervention of the developer.

Role in Prompting: Agent uses conversational context and enhanced prompting (including from voice-to-text) to comprehend the structure. When asked to build a customer-level dashboard, for example, Codex reads the prompt and works out a logical way to execute the task before changing any code.

Role in Pipelines: Codex generates data pipelines dynamically by writing the required dbt models and YAML configuration files. It works as the link between raw data in Codex and visualization output, running dbt build commands directly from the terminal to deploy to power bi and test data.

2.1 Use Case Diagram

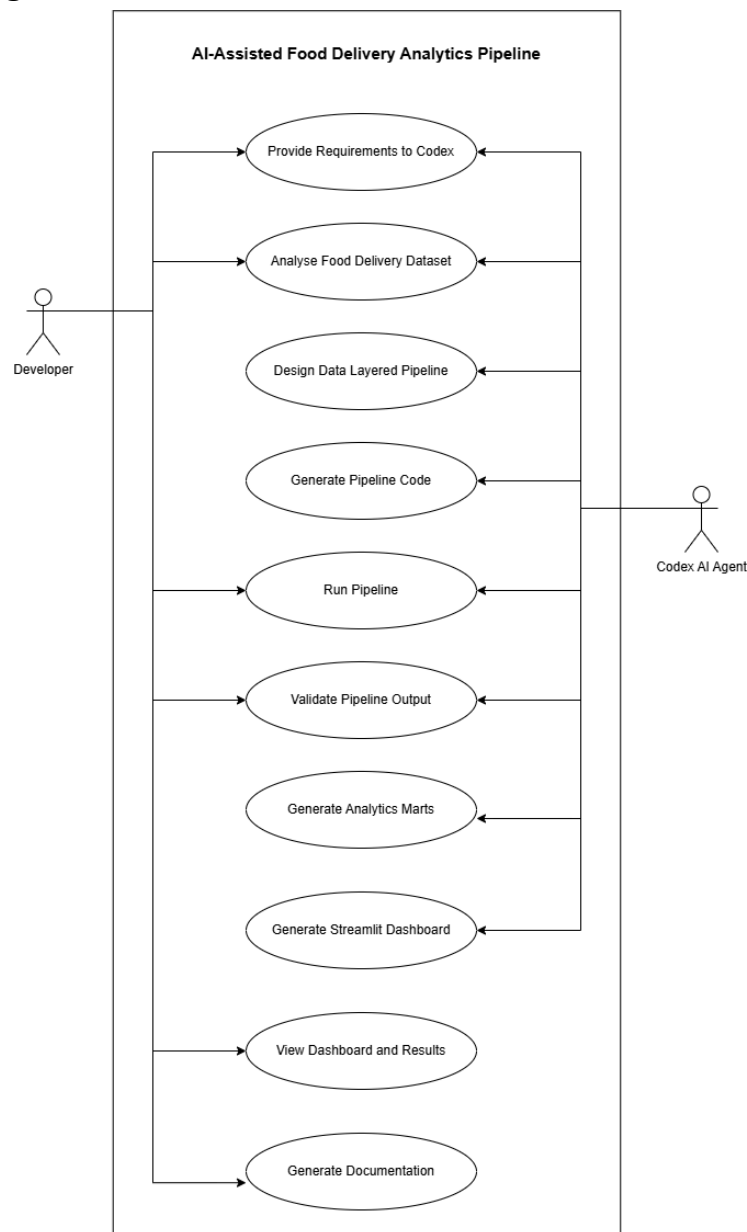


Figure 2.1 Use Case Diagram for the Food Delivery Analytics Pipeline using Codex

Figure 2.1 illustrates the use case diagram for the AI-Assisted Food Delivery Analytics Pipeline developed using Codex. The primary actor is the Developer, who provides project requirements and supervises the development process. Instead of manually implementing every component, the developer interacts with Codex AI Agent through prompts and reviews the generated outputs before execution.

Based on the provided requirements, Codex assists in analysing the food delivery dataset, designing the layered data pipeline, generating pipeline code, validating outputs, creating analytics marts, generating Streamlit dashboard components and producing project documentation. The developer remains responsible for reviewing the generated solutions, executing the pipeline and evaluating the final dashboard results.

This workflow demonstrates the concept of AI-assisted data engineering, where the engineer acts as a supervisor and decision maker while the AI agent performs repetitive development tasks. The collaboration between the developer and Codex improves productivity, reduces manual coding effort and accelerates the overall pipeline development process.

3.0 Steps to do prompt and coding

The pipeline was developed using an agentic AI-assisted workflow within Visual Studio Code (VS Code). The development process followed a YouTube tutorial that demonstrated how AI agents can be used to build a data engineering pipeline through prompts. Although the tutorial used Claude, Codex was chosen for this project because it was more suitable for handling many prompts and code generation tasks required throughout the development process. Codex was used to assist in dataset analysis, pipeline design, code generation, testing and dashboard development.

Step 1: Dataset Ingestion and Profiling

Codex was instructed to analyze the project repository and examine the raw dataset 'Food_Delivery_Times.csv'. Figure 3.1 shows the prompt requested Codex to identify the dataset structure, datatypes, important business columns and potential data quality issues.

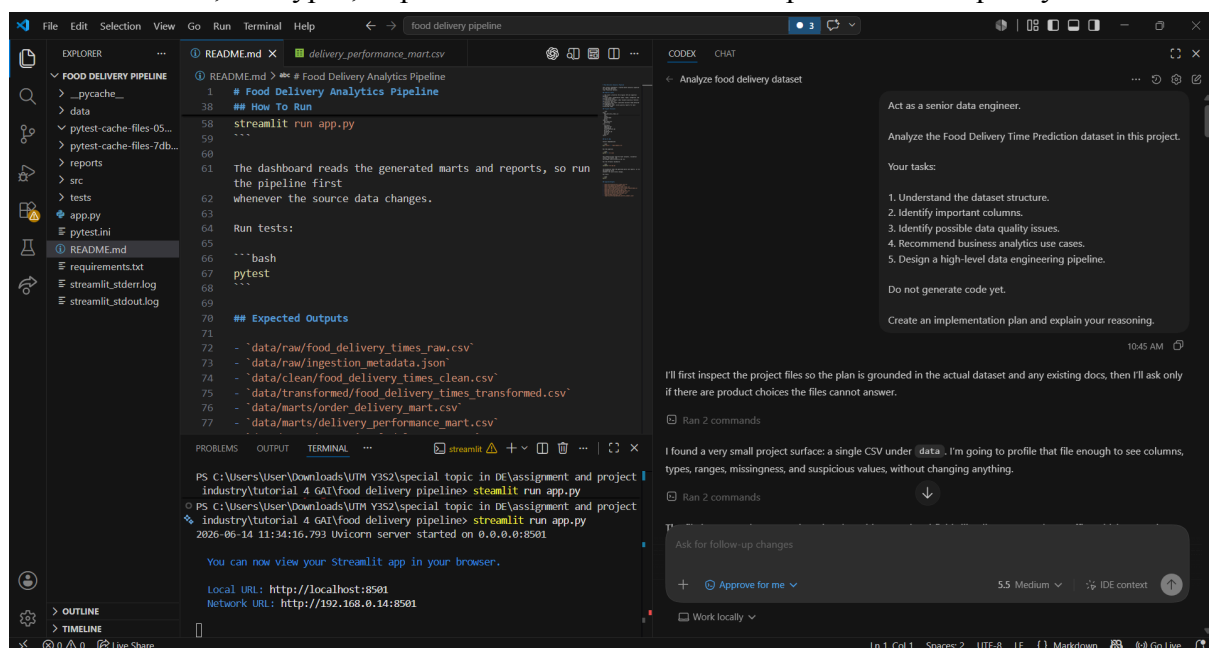


Figure 3.1 Prompt AI to analyze the dataset

Figure 3.2 shows the prompt to Codex for the data profiling process. Through this analysis, important columns such as 'Distance_km' and 'Delivery_time_min' were identified, while missing values and data inconsistencies were detected for further processing. In addition, Codex summarized the dataset characteristics, including the number of records, column distributions and missing value statistics. Based on these findings, Codex proposed a structured implementation plan for the data processing layers.

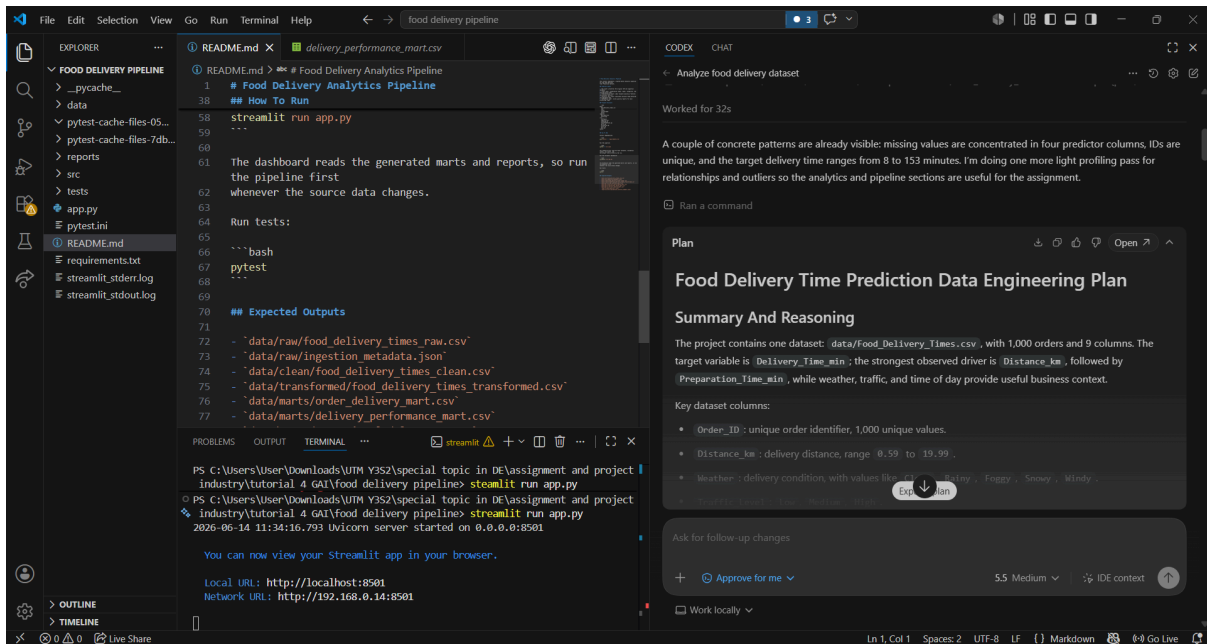


Figure 3.2 Prompt AI to profile the dataset

Step 2: Design Layered Pipeline

After understanding the dataset, Codex was instructed to design a complete analytical pipeline consisting of the Raw Layer, Clean Layer, Transformation Layer, Analytics Mart Layer and Validation Layer shown in Figure 3.3. This structure is important because it separates each stage of data processing, making the system more organised, scalable and easier to maintain.

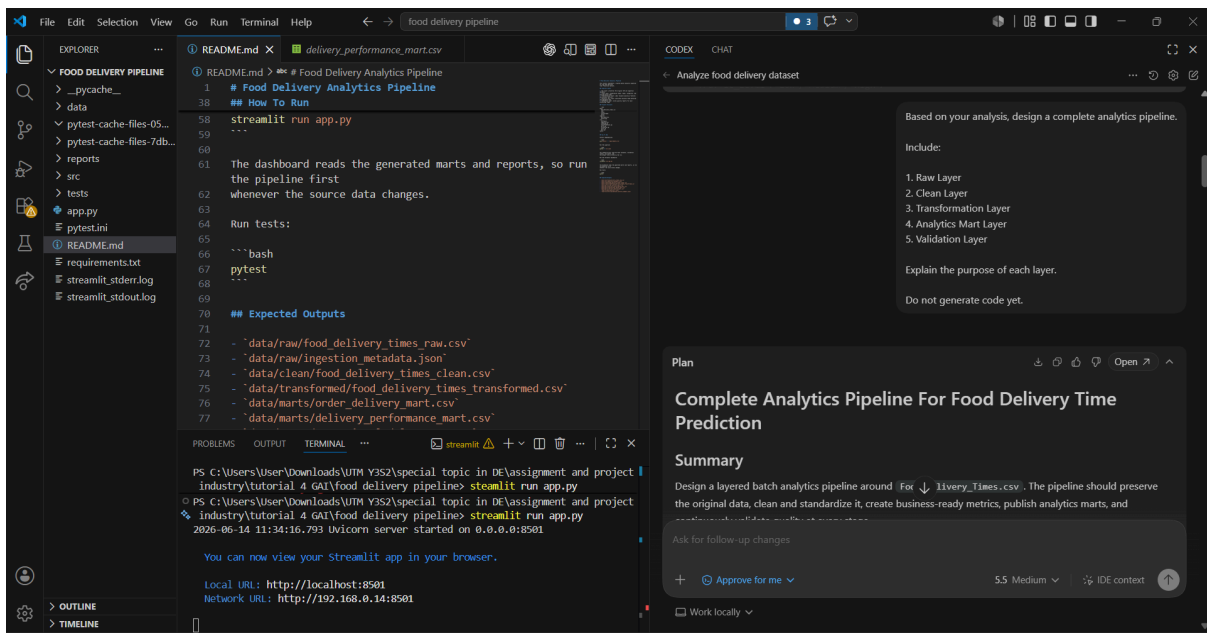


Figure 3.3 Prompt AI to design layered pipeline

Step 3: Project Structure Planning

Codex was then used to design the project structure before code generation began to improve scalability. Figure 3.4 shows the proposed project layout.

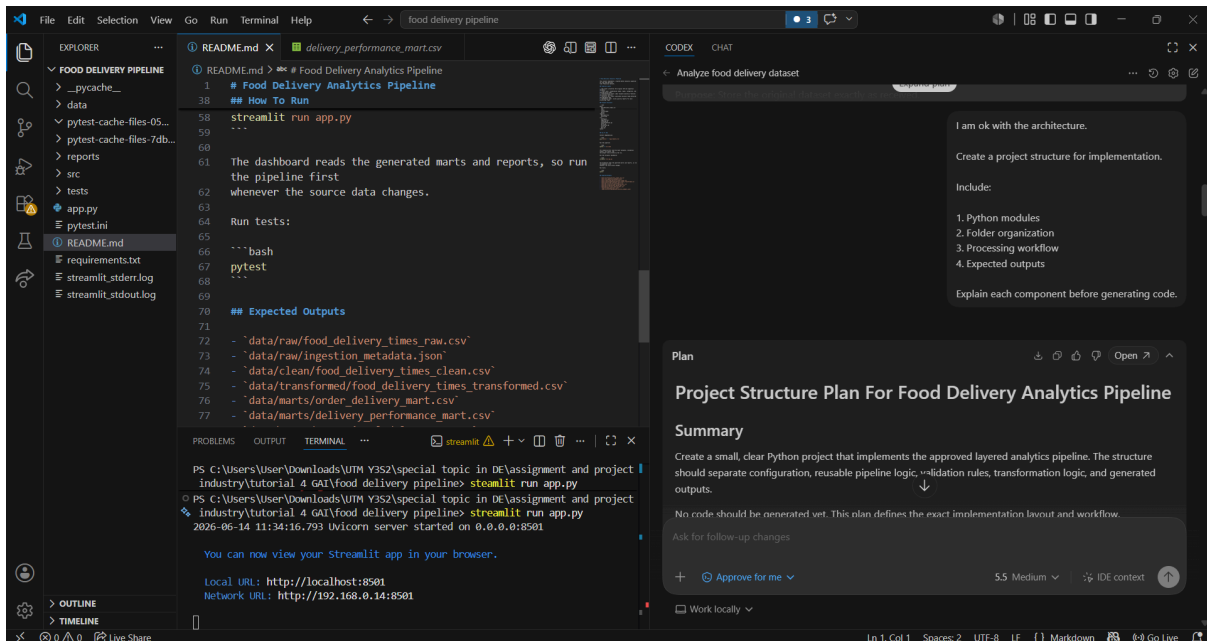


Figure 3.4 Prompt AI to create project structure

Step 4: Implementation and Code Verification

After the project structure was finalized, Codex was prompted to implement the supporting files. Figure 3.5 shows the automated code generation process, while Figure 3.6 presents the verification stage. During this step, Codex updated dependency requirements and executed validation commands such as 'python -m src.main' and 'pytest' to verify that the pipeline operated correctly.

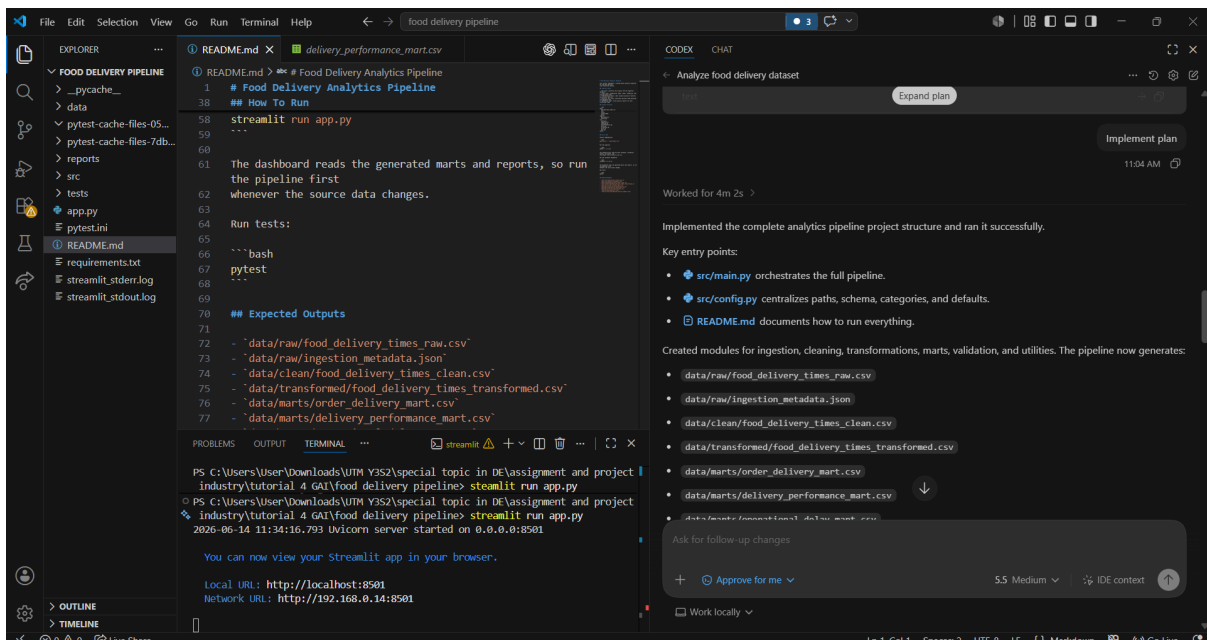


Figure 3.5 Prompt AI for implementation

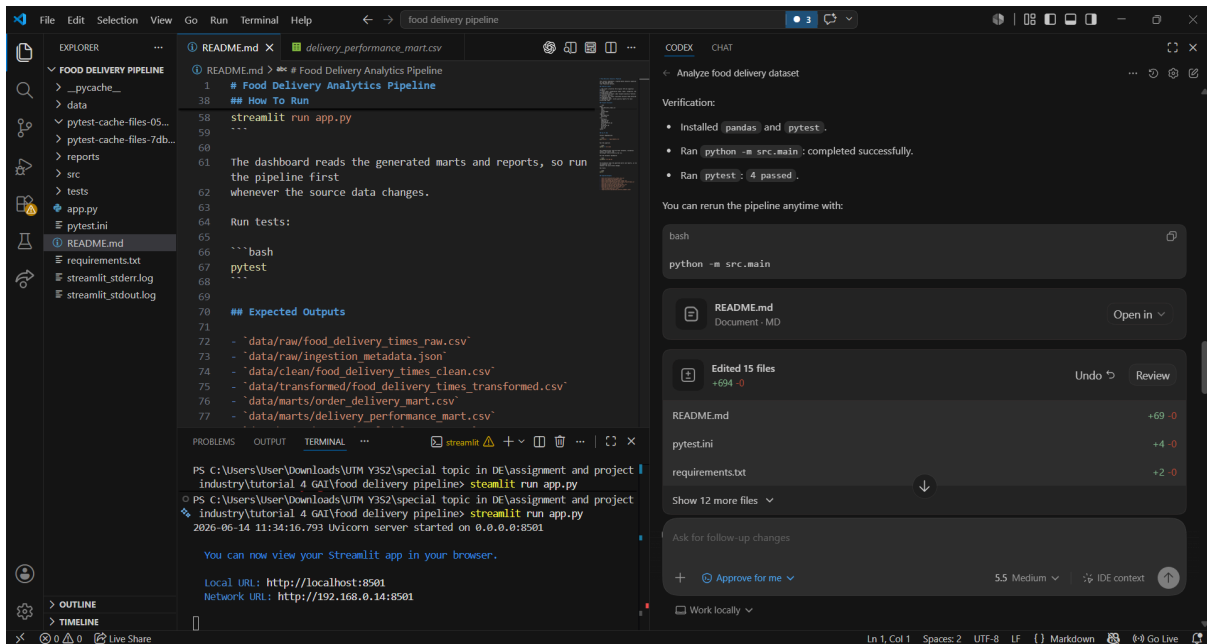


Figure 3.6 Verification stage

Step 5: Implementing Pipeline Progress Logs

Figure 3.7 shows Codex was prompted to add detailed execution logs for simplifying debugging activities.

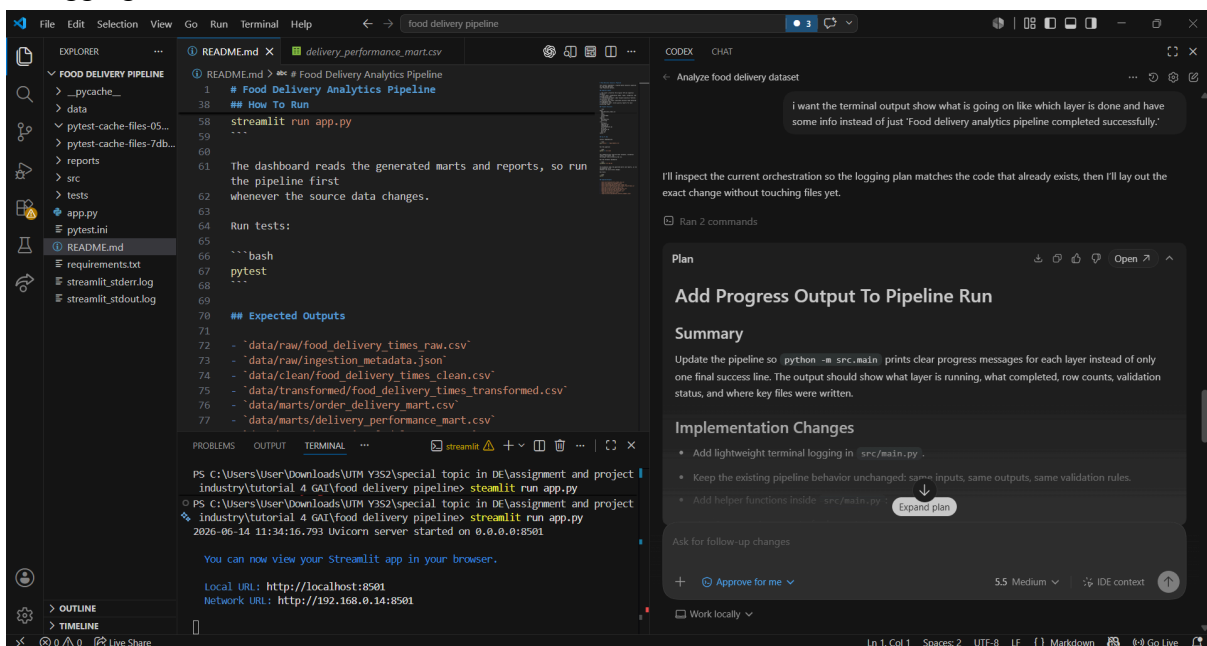


Figure 3.7 Prompt AI to implement pipeline progress logs

Figure 3.8 and Figure 3.9 shows the output of the pipeline. The output clearly displays each processing stage, the number of records processed, validation results and the generated output files. All validation checks passed with zero errors and the pipeline completed successfully. These logs provide clear visibility into the execution process and confirm that all layers of the data pipeline ran as expected.

```
● PS C:\Users\Teh\Downloads\food_delivery_pipeline_submission> python -m src.main
Starting Food Delivery Analytics Pipeline

[1/6] Setting up output folders...
Done: output folders ready

[2/6] Raw layer: ingesting source CSV...
Done: loaded 1000 rows and 9 columns
Output: data/raw/food_delivery_times_raw.csv
Output: data/raw/ingestion_metadata.json

Raw validation: checking schema and row count...
Validation: raw passed with 0 error(s) (1000 rows)
Output: reports/data_quality/raw_validation_report.json

[3/6] Clean layer: standardizing columns and handling missing values...
Done: clean dataset created with 1000 rows
Validation: clean passed with 0 error(s) (1000 rows)
Output: data/clean/food_delivery_times_clean.csv
Output: reports/data_quality/clean_validation_report.json

[4/6] Transformation layer: creating metrics and buckets...
Done: transformed dataset created with late-delivery and speed metrics
Validation: transformed passed with 0 error(s) (1000 rows)
Output: data/transformed/food_delivery_times_transformed.csv
Output: reports/data_quality/transformation_validation_report.json
```

Figure 3.8 Output of the pipeline

```
[5/6] Analytics mart layer: building business marts...
Done: created 4 marts
Validation: marts passed with 0 error(s)
Output: data/marts/order_delivery_mart.csv
Output: data/marts/delivery_performance_mart.csv
Output: data/marts/operational_delay_mart.csv
Output: data/marts/courier_vehicle_mart.csv
Output: reports/data_quality/mart_validation_report.json

[6/6] Reports: writing profiling report...
Done: reports written
Output: reports/profiling/dataset_profile_summary.json

Pipeline completed successfully.
```

Figure 3.9 Output of the pipeline

Step 6: Streamlit Dashboard Deployment

Codex was instructed to generate a visualization by using Streamlit. Figure 3.10 shows the plan proposed by Codex and Figure 3.11 shows the implementation of visualization.

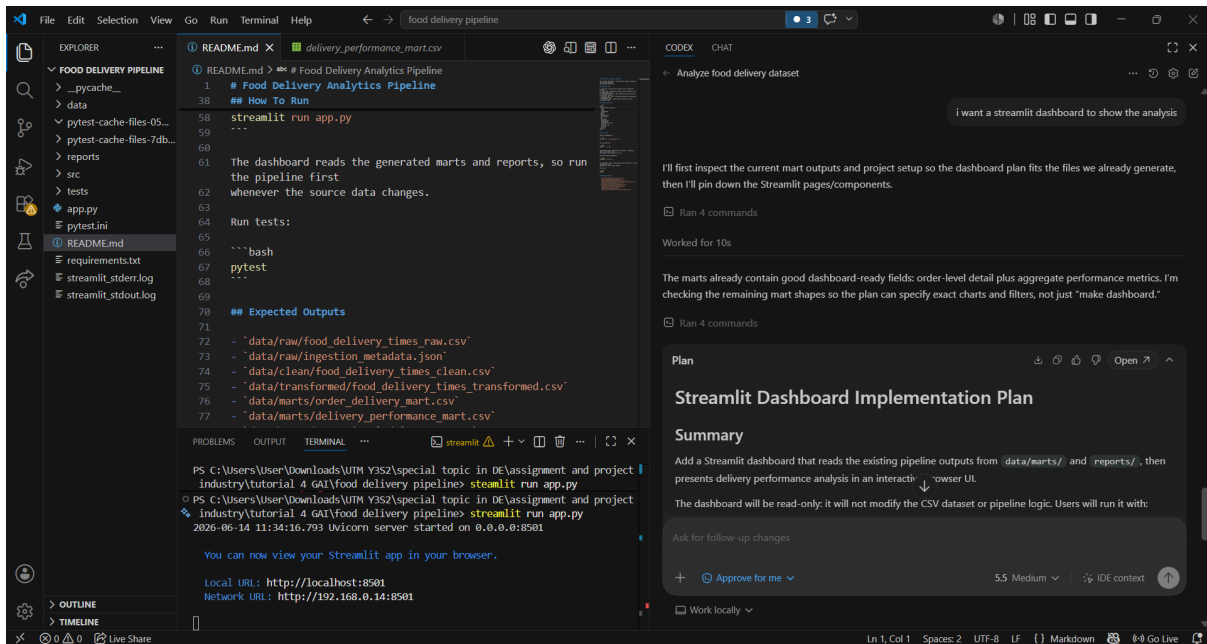


Figure 3.10 Dashboard implementation plan proposed by Codex

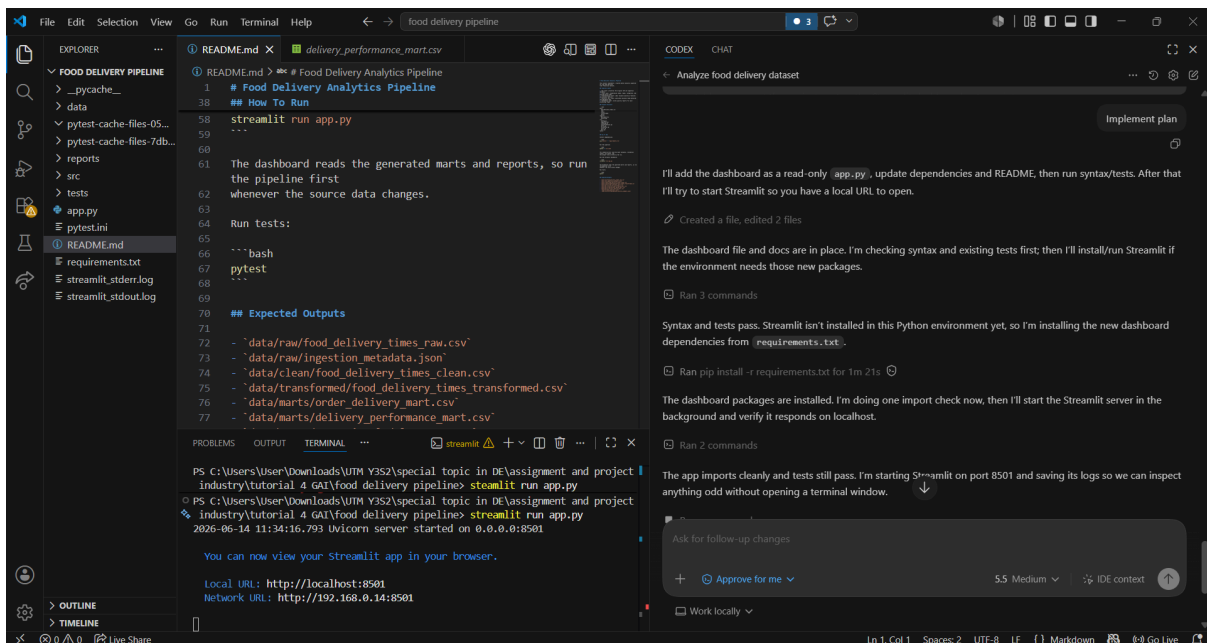


Figure 3.11 Implementation of dashboard

Figure 3.12 shows the dashboard visualization was successfully implemented.

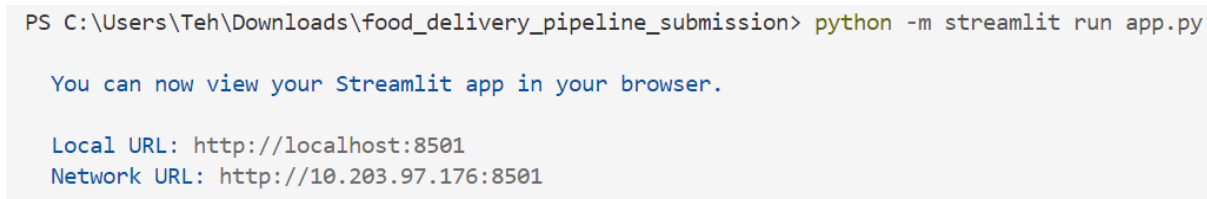


Figure 3.12 Output of dashboard visualization

Figure 3.13, 3.14, 3.15 shows the dashboard in the local host.

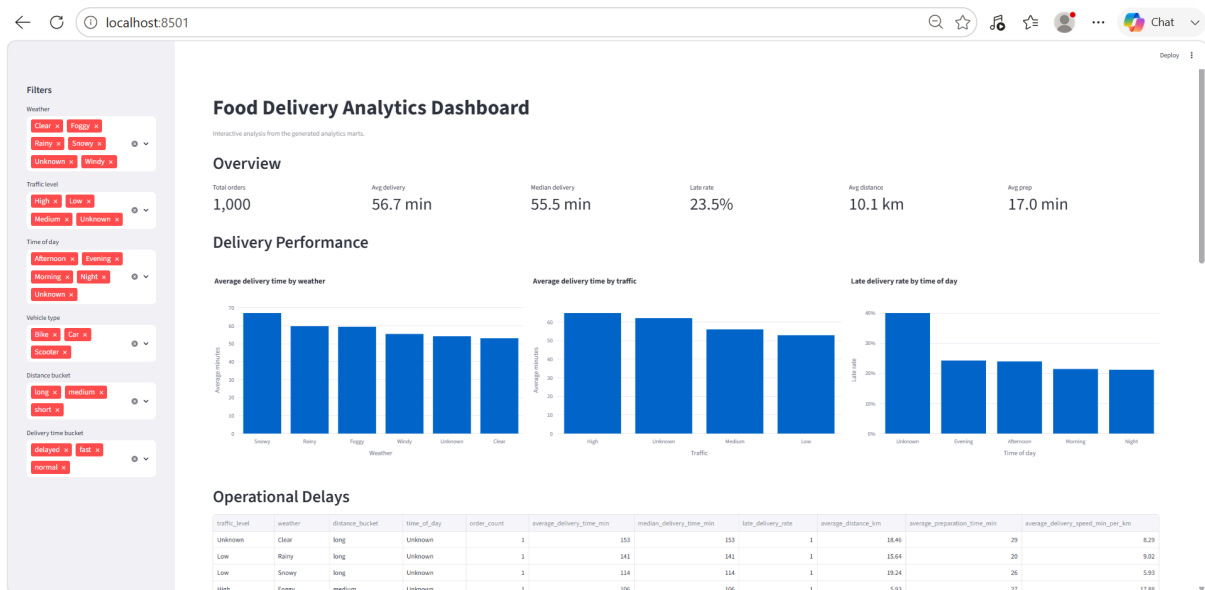


Figure 3.13 Dashboard

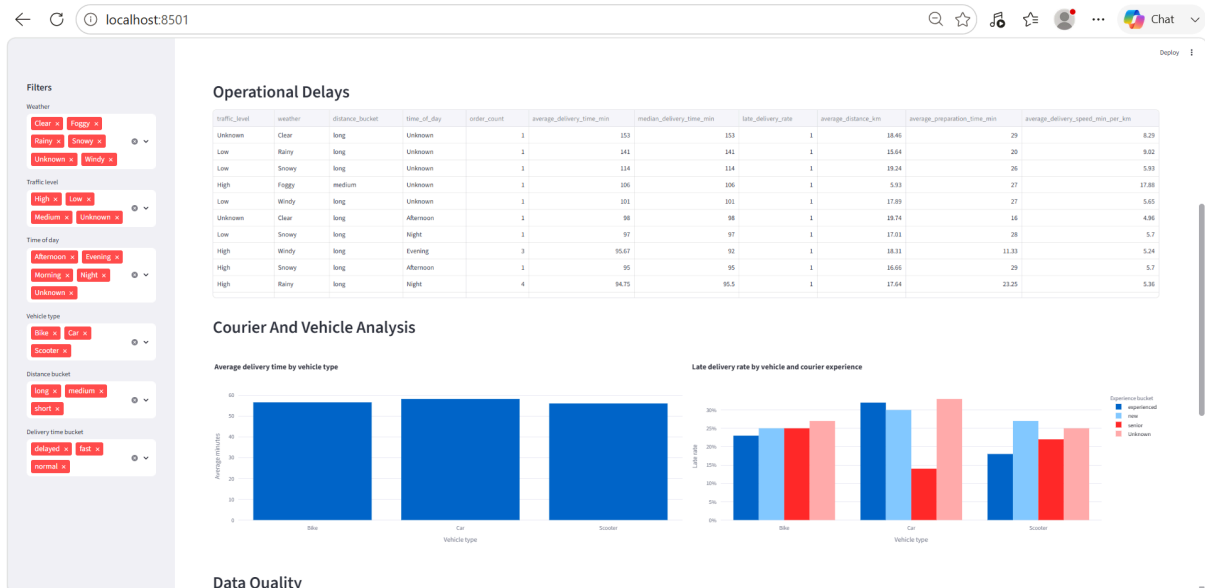


Figure 3.14 Dashboard

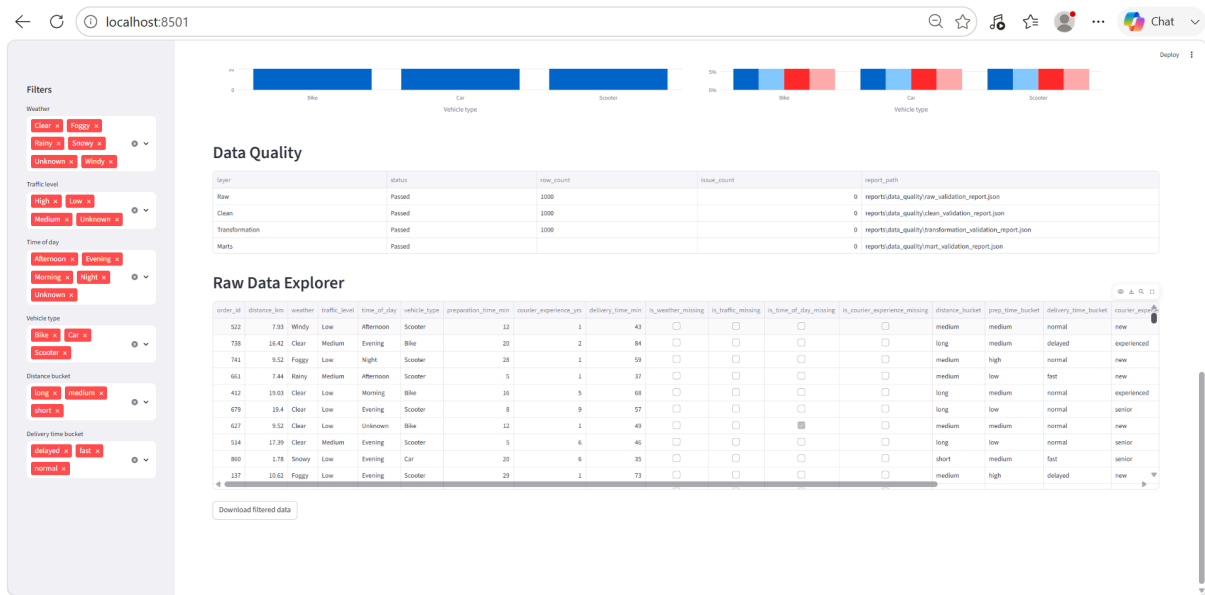


Figure 3.15 Dashboard

4.0 Reflection

LING YU QIAN

The role of a data engineer is fundamentally changed from being the one who writes the syntax to being the one who plans the pipeline and is the “coach” to the AI agent. The data engineer's job changes from writing the syntax to planning the pipeline and acting as “coach” of the AI agent. Automating repetitive tasks such as generating dbt models or complex Git version control workflows exponentially speeds up development with Codex, but it also has a lack of true domain context. This limitation is clearly illustrated in the tutorial when the agent tries to fix a dashboard error but instead changes unrelated drop-down logic because he doesn't realize that it is a simple local caching problem. To reduce these risks and improve future data pipelines, it's crucial to implement realistic improvements, including strict code prompt templates that ensure accurate data coding standards from the outset, and comprehensive automated data quality testing before the AI can openly make any recommendations.

TEH RU QIAN

This tutorial improved understanding of AI-assisted workflows and basic data engineering concepts. Experience was gained in building a full pipeline from raw data to clean data, transformation, analytics marts and dashboard using Codex. A key learning was that AI tools can speed up development, but they depend on clear and detailed instructions to produce correct results. One main challenge was prompt writing, where unclear prompts often led to incorrect outputs and required repeated adjustments. This showed that AI-assisted development is not fully automatic and needs proper planning and structured thinking. Overall, understanding changed from seeing AI as a simple code generator to viewing it as a helpful tool that works best when guided properly.

LIM YU HAN

The exploration of AI-assisted data engineering workflows demonstrates that the role of a data engineer is shifting from manual script writing to high-level architecture orchestration. Through the practical application of the Codex AI agent, significant insights were gained into how modern automated agents manage end-to-end data pipelines, from raw ingestion to the final generation of a Streamlit analytical dashboard. A major benefit identified is the efficiency gain in automating boilerplate code and staging layers, which allows development teams to bypass tedious syntax configurations and focus on data quality. However, a critical limitation observed is the AI's lack of inherent domain context, which occasionally leads to logical misalignments during complex data transformations if the initial instructions are ambiguous. To mitigate these limitations in future implementations, it is essential to establish highly structured, strict prompt templates and integrate automated data validation testing into the pipeline loop. Ultimately, this tutorial illustrates that while AI agents exponentially accelerate development speeds, human oversight and rigorous engineering structure remain vital to achieving reliable results.